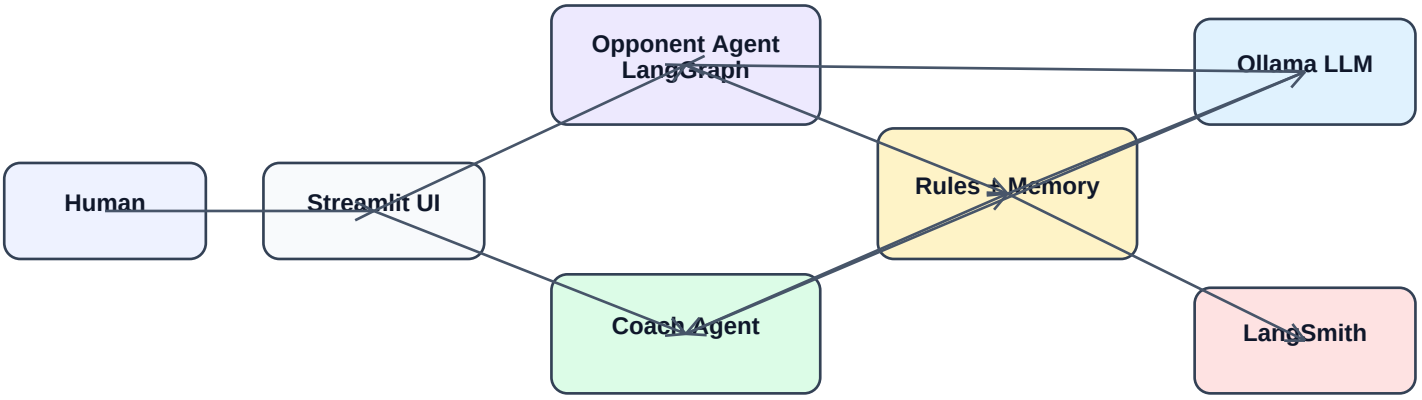
Bulls and Cows LangGraph Agent

Project Submission Document

An interactive agent demo using LangGraph, Ollama, Streamlit, and LangSmith observability.



Repository: github.com/ravikiran-uppalapati/bullcow

Primary local LLM: Ollama with gemma3:4b

Observability: LangSmith EU endpoint

1. Problem Statement

The project demonstrates how an agent can solve an iterative reasoning problem using memory, tools, state transitions, and observability. The selected problem is a 3-digit Bulls and Cows game where a human and an opponent agent take turns guessing secret numbers.

This is not a one-shot question-answer task. A player must guess, observe bulls/cows feedback, remember previous turns, eliminate impossible answers, and choose the next move. This makes it a good project for demonstrating an agentic reasoning loop.

Problem Requirements

- Maintain memory of all previous guesses and bulls/cows feedback.
- Use tools to score guesses and remove impossible numbers.
- Continue reasoning over multiple turns until a win or conflict.
- Explain suggestions to the human player.
- Make the internal decision process observable for project review.

2. High-Level System Architecture

The system combines deterministic game tools with agent workflows and LLM-powered conversation. Streamlit provides the game interface, LangGraph manages the opponent workflow, the Coach Agent helps the human player, Ollama gives natural-language responses, and LangSmith records the internal reasoning trace.

Component	Role
Streamlit UI	Interactive game screen, forms, Coach panel, history notebook, and winner dialogs.
Rules Engine	Validates numbers, generates 648 valid candidates, and scores bulls/cows.
LangGraph Opponent Agent	Maintains candidate state, receives feedback, filters candidates, and chooses guesses.
Coach Agent	Tracks human guesses, filters possible opponent secrets, suggests next moves, and explains why.
Session Memory	Stores agent turns, human turns, Coach chat, candidate counts, and a merged timeline.
Ollama LLM	Adds conversational personality and coaching using full game context.
LangSmith	Traces agent steps, state changes, human turns, and LLM messages.

3. LLM Used

For the local demo, the project uses Ollama with the locally installed gemma3:4b model. Ollama is selected because it runs locally, avoids hosted quota limits, and makes the demo reliable even when internet-hosted model providers are unavailable.

- Ollama is used for natural language only, not for hidden game logic.
- The strategic guess is produced by deterministic candidate filtering.
- Nebius and Gemini are supported as hosted fallbacks for Streamlit Cloud.
- A deterministic fallback remains available if no LLM provider is configured.

4. Agents in the System

Opponent Agent

The Opponent Agent guesses the human player's secret number. LangGraph manages its workflow: initialize state, generate a guess, receive feedback, filter candidates, choose the next guess, detect win/conflict, and explain reasoning.

Coach Agent

The Coach Agent helps the human player. It remembers previous guesses, bulls/cows responses, digits tried, remaining possible secrets, suggested next guess, and reasoning for the suggestion.

5. How the Problem Is Solved

The game is solved through candidate elimination. At the start there are 648 possible 3-digit secrets: 9 choices for the first digit, 9 choices for the second digit, and 8 choices for the third digit. After each feedback response, invalid candidates are removed.

For example, if the agent guesses 102 and receives 0 bulls and 1 cow, only numbers that would produce exactly that response against 102 remain in the candidate pool. This loop continues until the secret is solved or feedback becomes contradictory.

Requirement	Project Solution
Remember previous guesses	Session memory stores human turns, agent turns, Coach chat, and timeline.
Score bulls/cows accurately	Rules engine provides deterministic score_guess logic.
Reduce impossible answers	LangGraph opponent workflow and Coach Agent both filter candidate pools.
Explain next actions	Coach Agent produces suggested guess, possible count, and reasoning.
Avoid black-box behavior	LangSmith traces state, turns, LLM messages, and workflow execution.

6. Why These Design Decisions Were Made

- LangGraph was chosen because the opponent requires an explicit reasoning workflow, not a single chatbot response.
- Ollama was chosen for local reliability, no hosted quota dependency, and simple laptop demonstrations.
- LangSmith was chosen to make the internal agent loop visible and debuggable.
- Streamlit was chosen for a fast interactive UI suitable for demos and project review.
- The rules engine remains deterministic so the game can be tested and explained.

System	Decision Rationale
LangGraph	Multi-step reasoning workflow with state, feedback, branching, and final status detection.
Ollama	Primary local LLM to avoid free-tier quota issues and make the local demo reliable.
LangSmith	Observability, traceability, and explainability of the agent loop.
Streamlit	Fast interactive demo UI suitable for project submission and walkthrough.
Rules Engine	Correct, testable game logic kept separate from LLM-generated language.

7. Observability With LangSmith

LangSmith is the observability layer. It records LangGraph node execution, human guesses, agent feedback turns, LLM messages, and the state passed into each step. This helps prove the agent was reasoning over memory and tools rather than guessing randomly.

8. Testing and Validation

The project includes 49 passing automated tests covering:

- Bulls/cows scoring and candidate generation.
- Candidate filtering and LangGraph state updates.
- Coach notes, possible-count reasoning, and notebook rendering.
- Session memory and LLM prompt construction.
- Settings for Ollama, Nebius, Gemini, and LangSmith.

9. Demo Flow

- Start the Streamlit app.
- Think of a valid 3-digit secret number.
- Let the Opponent Agent guess and provide bulls/cows feedback.
- Make human guesses and watch the Coach Agent track history.
- Ask the Coach a question and show that Ollama uses full game memory.
- Open LangSmith to show traces for LangGraph, turns, and LLM messages.

10. Limitations and Future Improvements

- Local Ollama works only where Ollama is running; Streamlit Cloud needs Nebius or Gemini.
- The guessing strategy is explainable but not optimized for the absolute fewest guesses.
- Future work could add minimax-style guessing, richer trace links, and UI controls for provider selection.

11. Conclusion

This project demonstrates a complete agent pattern: LangGraph controls the reasoning workflow, deterministic tools provide reliable game logic, the Coach Agent supports the human with memory and explanation, Ollama adds conversational intelligence, Streamlit provides interaction, and LangSmith makes the process observable.